

Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
First Semester 2024-2025

Mid-Semester Test
(EC-2 Regular)

Course No. : AIMLCZG511
Course Title : Deep Neural Network
Nature of Exam : Closed Book
Weightage : 30%
Duration : 2 Hours
Date of Exam : 29-06-2025 (FN)

No. of Pages	= 03
No. of Questions	= 06

Note to Students:

1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. Assumptions made if any, should be stated clearly at the beginning of your answer.

Q.1 Consider the implementation of a perceptron for a NAND gate using bipolar representation (i.e., input and output values are either +1 or -1). Assume the following: [5 Marks]

- Initial weights: $w_0=w_1=w_2=0$, where w_0 is the bias and its input $x_0=1$
- Learning rate: $\eta=1$
- Activation function: $h=\text{sign}(z)$, where $z=w_0+w_1x_1+w_2x_2$
- Output: $y=h=\text{sign}(z)$
- Weight update rule: $\Delta w_i=\eta(t-y)x_i$, and $w_i \leftarrow w_i+\Delta w_i$

X_1	X_2	Target
-1	-1	1
-1	1	1
1	-1	1
1	1	-1

- A. Implement the perceptron learning algorithm for the above NAND gate using one full epoch (i.e., update the weights based on all four input combinations once).
- B. Show the computations for z , y , Δw , and updated weights for each input.
- C. Write the final weight vector (w_0 , w_1 , w_2) after the epoch.
- D. Verify that the perceptron correctly classifies all inputs with the final weights.

Note: Answer should be presented in tabular form similar to the example discussed in class.

Answer Key:

X1	X2	T	Z	Y = SIGN(Z)	$\Delta W_0 =$ $H(T-Y)X_0$	$\Delta W_1 =$ $H(T-Y)X_1$	$\Delta W_2 =$ $H(T-Y)X_2$	UPDATED (W0, W1, W2)
-1	-1	1	$0 + 0 \times (-1) + 0 \times (-1) = 0$	1	0	0	0	(0, 0, 0)
-1	1	1	$0 + 0 \times (-1) + 0 \times 1 = 0$	1	0	0	0	(0, 0, 0)
1	-1	1	$0 + 0 \times 1 + 0 \times (-1) = 0$	1	0	0	0	(0, 0, 0)
1	1	-1	$0 + 0 \times 1 + 0 \times 1 = 0$	1	-2	-2	-2	(-2, -2, -2)

Final weight vector after epoch

$(w_0, w_1, w_2) = (-2, -2, -2)$

Check each point

- $(-1, -1)$: $z = -2 + (-2)(-1) + (-2)(-1) = -2 + 2 + 2 = 2 \Rightarrow \text{sign}(2) = 1$
- $(-1, 1)$: $z = -2 + (-2)(-1) + (-2)(1) = -2 + 2 - 2 = -2 \Rightarrow \text{sign}(-2) = -1$
- $(1, -1)$: $z = -2 + (-2)(1) + (-2)(-1) = -2 - 2 + 2 = -2 \Rightarrow \text{sign}(-2) = -1$
- $(1, 1)$: $z = -2 + (-2)(1) + (-2)(1) = -2 - 2 - 2 = -6 \Rightarrow \text{sign}(-6) = -1$

Q2. Consider a binary classifier using a perceptron to classify a set of 2D points into two classes: red and blue. **[5 Marks]**

After plotting the data, you observe the following:

- Red and blue points cannot be separated by any straight line in 2D space.
- Multiple curved or combined boundaries could separate the classes successfully.

Answer the following:

- Why would a single-layer perceptron fail to classify this dataset correctly? [1 mark]
- How can a Multi-Layer Perceptron (MLP) overcome this limitation? Describe the role of hidden layers and non-linear activation functions. [2 Mark]
- For an XOR function over 6 binary input variables, how many perceptrons are required if you use a single hidden layer network? how many perceptrons are needed in a deep network? [2 Mark]

Note: Answer/Justify/Illustrate in no more than 30-50 words and a precise response w.r.t the given question only. Vague answers will be penalized.

Answer Key:

- Single-layer perceptron limitation - a single-layer perceptron can only form linear decision boundaries. Since the data is not linearly separable, it fails to classify the points correctly. (1M)

B. MLP solution and hidden layers - An MLP uses hidden layers with non-linear activation functions (e.g., ReLU, sigmoid), allowing it to learn non-linear and complex decision boundaries, enabling separation of non-linearly separable data. (2M)

C. XOR with 6 inputs (2M)

- Single hidden layer: Needs at least $2^6 = 64$ perceptrons in hidden layer to represent all combinations.
- Deep network: Can be constructed with fewer perceptrons per layer using multiple hidden layers hierarchically.

Clarification for C (if needed)

- XOR is not linearly separable; each added variable doubles combinations.
- Deep networks can decompose logic progressively (e.g., using 2-variable XORs repeatedly).

Q3. A medical diagnostics company is developing two AI models: **[4 Marks]**

- Model A uses a neural network to detect if a patient has COVID-19 or not.
- Model B classifies patients into one of five disease categories based on symptoms.

Based on this context:

- Justify the use of a shallow neural network or a deep network for these models, respectively.
- Which loss function is appropriate for each system? Briefly justify your choices.
- What is the role of the SoftMax activation in Model B's output layer?
- Can SoftMax be used in System A? Explain why or why not.

Note: Answer/Justify/Illustrate in no more than 30-50 words and a precise response w.r.t the given question only. Vague answers will be penalized.

Answer Key:

A. Network choice justification (0.5M+0.5M)

- Model A: Shallow network suffices (binary classification, simpler decision boundary).
 - Model B: Deep network preferred (complex multi-class problem, high-level features needed).
-

B. Appropriate loss function (0.5M+0.5M)

- Model A: Binary Cross-Entropy — suitable for two-class (0/1) problems.
 - Model B: Categorical Cross-Entropy — handles multi-class outputs effectively.
-

C. Role of SoftMax in Model B (1M)

SoftMax converts logits to class probabilities, ensuring outputs sum to one, aiding multi-class decisions.

D. SoftMax in System A (0.5M+0.5M)

No, System A uses a sigmoid for binary output (probability of one class). SoftMax is redundant for two classes.

4. An industrial AI model is trained to predict energy consumption from sensor readings using a linear transformation. The model computes the following function: **[6 Marks]**

$$f(x, W) = \|W^\top x\|^2 = \sum_{j=1}^m (W^\top x)_j^2$$

Where:

- $x \in \mathbb{R}^2$ is the sensor input vector
- $W \in \mathbb{R}^{2 \times 2}$ is the weight matrix

Given:

$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}, \quad x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

Answer the following:

- Draw a computational graph that clearly illustrates the operations from x , W to the scalar output $f(x, W)$.
- Perform the forward pass to compute the intermediate vector $q = W \cdot x$ and the final output f .
- Compute the gradient of the loss function for: $\nabla_q f$, $\nabla_W f$, $\nabla_x f$ (Show full matrix steps.)
- Explain in 2–3 lines how the computed gradients would be used to update W and x in a neural network training step.

Answer key:

Part	Answer Summary	Marks
A	Graph showing $x, W \rightarrow q \rightarrow f$	1
B	$q = [0.22, 0.26]^T, f = 0.48$	1
C	$\nabla_q = [1, 1], \nabla_W = [[0.2, 0.4], [0.2, 0.4]], \nabla_x = [-0.2, 1.3]^T$	3
D	Explanation on parameter updates	1

Q5. Consider the following Python code snippet used to train a deep neural network on a multiclass classification task with 5 output classes, using the Keras library: **[5 Marks]**

```
model = Sequential()
model.add(Dense(64, input_dim=30, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='SGD', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=16)

test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")
```

Answer the following questions concisely:

- A. Describe the neural network architecture and the purpose of the activation functions used in each layer.
- B. Explain why categorical cross-entropy is appropriate for this setup. How would it change if labels were not one-hot encoded?
- C. What does a high training accuracy and low test accuracy suggest? Define overfitting and generalization in this context.
- D. Justify the use of the RMSprop optimizer. Mention one benefit and an alternative optimizer suitable for this task.
- E. Suggest two techniques to improve model generalization or training stability.

Note: (Answers must be concise within 30–50 words. Justifications are required for full credit.)

Answer Key:

A. Architecture & activations (1M)

Two hidden layers (64 and 32 units) with ReLU (introduces non-linearity, prevents vanishing gradients), final layer with sigmoid (used for binary; for multiclass, should be softmax to output class probabilities).

B. Loss function choice (1M)

Categorical cross-entropy compares predicted probability distributions with one-hot encoded labels, penalizing wrong class probabilities.

If labels are integers (not one-hot), use sparse categorical cross-entropy.

C. Accuracy interpretation (1M)

High training accuracy, low test accuracy $\Rightarrow$ overfitting (model memorizes training data but fails to generalize).

Overfitting: Poor performance on unseen data.

Generalization: Ability to perform well on new data.

D. Optimizer justification (1M)

RMSprop adapts learning rates per parameter, stabilizing training on noisy or non-stationary problems.

Benefit: Faster convergence.

Alternative: Adam (adaptive learning rate + momentum).

E. Improve generalization (1M)

- Add dropout layers (prevent co-adaptation of neurons).
- Use early stopping (stop training when validation loss stops improving).

The code shown actually uses sigmoid output with `binary_crossentropy`, which is correct for binary classification. For multiclass, the final layer should be softmax with `categorical_crossentropy`. This can be used to assess conceptual understanding.

Q6. Answer the following questions:

[5 Marks]

- A. You want to solve a classification task. You first train your network on 20 samples. Training converges, but the training loss is very high. You then decide to train this network on 10,000 examples. Is your approach to fixing the problem correct? If yes, explain the most likely results of training with 10,000 examples. If not, give a solution to this problem.
- B. A deep neural network for character recognition uses tanh in hidden layers and trains slowly with vanishing gradients. A switch to ReLU improves convergence.
- I. Briefly explain why tanh caused issues.
 - II. Why does ReLU work better in this scenario?
- C. A fully connected neural network has 8 input neurons, two hidden layers with 10 and 6 neurons, and an output layer with 4 neurons. Biases are not included. Compute the total number of trainable weights.

Note: (Answers must be concise within 30–50 words. Justifications are required for full credit.)

Answer Key:

A. Data size and loss (1.5M)

Yes, correct approach - With only 20 samples, the model underfits due to insufficient data. Using 10,000 samples allows the model to learn better patterns, reducing training loss and improving generalization.

B. Activation function choice (2M)

I. Why tanh causes issues - tanh squashes outputs between -1 and 1 , causing gradients to vanish in deep networks, slowing or stopping learning.

II. Why ReLU works better- ReLU avoids saturation in positive region, maintains stronger gradients, accelerates convergence, and reduces vanishing gradient problem.

C. Total trainable weights calculation (1.5M)

- Input to first hidden: $8 \times 10 = 80$
 - First hidden to second hidden: $10 \times 6 = 60$
 - Second hidden to output: $6 \times 4 = 24$
- Total weights: $80 + 60 + 24 = 164$.
